

GARIL AI

Defensive Security Audit Report

Governed AI for Research, Instruction and Learning

Assessment type: Defensive / hardening review (not an offensive penetration test)

Scope: Next.js frontend + Fastify API + MongoDB + optional S3/MinIO

Initial review: 28 September 2026

Remediation update: 3 October 2026

Classification: Confidential — internal use

This report identifies security weaknesses, remediations, and fix status.
It does not include exploits, attack payloads, or reproduction procedures.

Overall residual risk: MEDIUM (17 fixed, 7 partial, 2 open)

1. Executive summary

The September 2026 defensive review rated overall risk Critical because unauthenticated user-management and related open API surfaces could list/create/update/delete users, expose sessions/outputs, and invoke costly AI without login. Governance/admin and portal stacks already had real authorization work, but legacy open routes undermined that model.

Remediation pass (3 October 2026) implemented P0 and most P1 hardening. Findings: 4 Critical, 9 High, 9 Medium, 4 Low — of which 17 Fixed, 7 Partial, 2 Open.

Residual risk is Medium: Critical open-API issues are closed. Remaining work is mainly httpOnly cookie sessions, token revocation, upload AV/MIME hardening, prompt-injection isolation, backup encryption, and CSP at the edge.

2. Project overview

- Frontend: Next.js 16 (React 19); client auth via localStorage Bearer JWT (cookie migration still open)
- Backend: Fastify 5 (backend/src/server.ts + portal routes) with default-deny API auth
- Database: MongoDB via Mongoose
- Object storage: Optional S3-compatible (MinIO)
- AI: OpenRouter + literature APIs (OpenAlex, PubMed, Tavily, etc.)
- Auth: Custom HS256 JWT (24h TTL); Resend for password-reset email

3. Positive controls in place

- Password hashing with scrypt + salt + timingSafeEqual
- JWT HMAC-SHA256 with AUTH_SECRET required outside local development
- Password-reset tokens hashed (SHA-256), 1-hour TTL, cleared after use; non-enumerating responses
- Default-deny /api/* middleware with explicit public allowlist
- Admin APIs: requireAdminScope / requireSuperAdmin / feature + university scoping + active status
- Portal APIs: requirePortalActor + ownership/tenant filters
- Per-user ChatSessionRegistry; WebSocket first-message auth (no JWT in URL)
- @fastify/rate-limit (global + auth/AI) and @fastify/helmet security headers
- HTML/markdown sanitization (DOMPurify / rehype-sanitize)
- Research assets generally filtered by userId; path traversal checks on outputs
- .env files gitignored; LLM/S3/mail secrets documented as backend-only
- Governance: audit logs, retention/deletion APIs, registration gated to onboarded universities

4. Findings and remediation status

Sorted by severity (Critical! Low). Status: Fixed (closed in code), Partial (mate

CRITICAL · A1 · Unauthenticated user CRUD API

Status: Fixed

Location: backend/src/server.ts (legacy /api/users*)

Description: GET/POST /api/users and PATCH/DELETE /api/users/:id had no Authorization checks. createUser accepted a role from the request body, enabling account lifecycle abuse and privilege escalation outside the protected admin user APIs.

Remediation: Remove these legacy routes, or gate every method with requireAdminScope / requireFeatureAccess. Never allow role assignment without the same rules as admin user creation.

Resolution: Legacy /api/users* routes return HTTP 410. User management is only via scoped /api/admin/users*.

CRITICAL · A2 · Default admin bootstrap can reset a weak known password

Status: Fixed

Location: backend/src/constants/default-admin.ts; bootstrap-admin.service.ts

Description: Unless DEFAULT_ADMIN_ENABLED=false, startup ensured an admin existed and could overwrite the password hash to match configured/default credentials.

Remediation: Opt-in bootstrap only; never reset passwords on mismatch; env-on first run.

Resolution: Create-only when account missing; never overwrites existing password. Password required from DEFAULT_ADMIN_PASSWORD. Production warns to disable after bootstrap.

CRITICAL · B1 · Unauthenticated dashboard, sessions, and outputs

Status: Fixed

Location: backend/src/server.ts (dashboard/sessions/outputs/status/chat)

Description: Dashboard stats/sessions, session messages, outputs, status, session reset, and chat abort lacked authentication.

Remediation: Require authentication; scope by userId; restrict cross-user dashboards to admins.

Resolution: Default-deny auth middleware; dashboard requires admin scope; session messages ownership-checked; chat/status/reset/abort require auth.

CRITICAL · J1 · Unauthenticated user/session listing exposes PII

Status: Fixed

Location: /api/users, /api/dashboard/sessions, session messages

Description: Names, emails, institutions, roles, and research session content may have been readable without login — severe privacy/GDPR risk if internet-exposed.

Remediation: Immediate auth lock; review access logs if routes were publicly reachable.

Resolution: Routes locked or removed. Operators should still review historical access logs if the API was internet-exposed before the fix.

HIGH · A3 · JWT stored in localStorage

Status: Partial

Location: lib/auth.ts; hooks/useAuth.tsx

Description: Bearer JWT in localStorage (feynman_auth_token). Any XSS can exfiltrate the session. No httpOnly / Secure / SameSite cookie session.

Remediation: Prefer httpOnly Secure SameSite cookies (or BFF); short-lived access tokens with refresh rotation; clear sessions on logout.

Resolution: Access token TTL reduced to 24 hours; XSS mitigated via HTML/markdown sanitization. Full httpOnly cookie/BFF session remains open (P2).

HIGH · A4 · WebSocket token in URL query string

Status: Fixed

Location: lib/api.ts; backend/src/server.ts (WS); hooks/useGarilSocket.ts

Description: Auth tokens in query strings appear in access logs, proxies, and history. WS also accepted connections without a token.

Remediation: Authenticate via first-message auth; reject unauthenticated sockets; never put JWTs in URLs.

Resolution: Client no longer appends ?token=. Server requires { type: "auth", token } within 10s; unauthenticated sockets are closed.

HIGH · A5 · Single global ChatService shared across clients

Status: Fixed

Location: backend/src/lib/chat-registry.ts; chat.service.ts

Description: One in-memory chat session served the process. Concurrent users could interfere with each other's session, abort, and status streams.

Remediation: Per-user chat sessions; require auth before subscribe/prompt/reset/abort.

Resolution: ChatSessionRegistry isolates ChatService instances per authenticated userId.

HIGH · B2 · AI generation endpoints callable without auth

Status: Fixed

Location: /api/research/outline, /ideas/generate, /api/papers/search

Description: When no Bearer token was present, outline/ideas still ran without feature gates or token deduction — anonymous LLM/API cost abuse.

Remediation: Require auth on all LLM/literature routes; enforce feature flags and quotas; add rate limits.

Resolution: Auth required via default-deny + requireAuthUser; feature flags and student token deduction enforced; AI routes rate-limited.

HIGH · B3 · Full-database backup for any console admin

Status: Fixed

Location: admin-backup.service.ts; server.ts backup routes

Description: Backup used requireAdmin (any console admin), dumped all collections with no tenant filter — university admins could obtain platform-wide PII.

Remediation: Restrict to requireSuperAdmin; tenant-scoped exports if needed later.

Resolution: All /api/admin/backup* routes now require requireSuperAdmin.

HIGH · C1 · Hardcoded default admin credentials in source

Status: Fixed

Location: backend/src/constants/default-admin.ts

Description: Default email/password existed in repository and docs examples — a likely bootstrap credential for anyone with repo access.

Remediation: Delete password constants; env-only bootstrap; fail closed if unset.

Resolution: Password constant removed. Docs/examples no longer show weak sample passwords.

HIGH · F1 · Default CORS reflects any origin

Status: Fixed

Location: backend/src/server.ts

Description: When CORS_ORIGIN was unset, origin: true allowed any origin.

Remediation: Require explicit CORS_ORIGIN allowlist in production; fail startup if unset.

Resolution: Production startup throws if CORS_ORIGIN is missing.

HIGH · G1 · No API rate limiting

Status: Fixed

Location: backend (@fastify/rate-limit)

Description: Login, password reset, registration, and LLM endpoints lacked throttling.

Remediation: Add @fastify/rate-limit (global + stricter on /api/auth/* and AI routes).

Resolution: Global 300/min; auth login/register 10/min; forgot-password 5/min; AI routes 20–30/min.

HIGH · I1 · Missing auth on numerous production API routes

Status: Fixed

Location: backend/src/lib/public-api.ts; server.ts onRequest hook

Description: Parallel open APIs beside hardened admin/portal APIs — leftover attack surface.

Remediation: Default-deny middleware: all /api/* except an allowlist require auth.

Resolution: Public allowlist: health, auth register/login/forgot/reset, universities catalogue, public legal. Everything else requires an active session.

MEDIUM · A6 · Long-lived JWT (7 days) without revocation

Status: Partial

Location: backend/src/lib/auth-token.ts; require-auth.ts

Description: Tokens remained valid until expiry. Password change / suspend / role change may not invalidate existing JWTs.

Remediation: Shorter access TTL; refresh tokens; tokenVersion or denylist; enforce status === active on every request.

Resolution: TTL reduced to 24h; active status enforced on every API request and admin load. Refresh tokens / revocation list still open (P2).

MEDIUM · A7 · Password-reset email enumeration

Status: Fixed

Location: auth.service.ts; /api/auth/forgot-password

Description: Missing accounts returned a distinct error revealing whether an address was registered.

Remediation: Always return the same 200 message; rate-limit by IP.

Resolution: Same success message for all emails; forgot-password rate-limited to 5/min.

MEDIUM · C2 · Fallback AUTH_SECRET outside production

Status: Fixed

Location: backend/src/config/env.ts

Description: Fixed development secret used when unset outside production — risk if staging ran without NODE_ENV=production.

Remediation: Fail if AUTH_SECRET missing unless explicitly development; refuse weak/default secrets in production.

Resolution: Fallback only when NODE_ENV=development. Production rejects missing or default AUTH_SECRET.

MEDIUM · D1 · HTML/markdown rendered without sanitization

Status: Partial

Location: portal editors; ResearchDocEditor; ReactMarkdown usages

Description: User- or model-generated HTML injected into the DOM enables XSS, which threatens JWT theft via localStorage.

Remediation: Sanitize with DOMPurify / rehype-sanitize; CSP script-src allowlist.

Resolution: isomorphic-dompurify on dangerouslySetInnerHTML paths; rehype-sanitize on ReactMarkdown. App-level CSP still optional at CDN/nginx.

MEDIUM · E1 · Presigned uploads: limited server MIME/malware checks

Status: Open

Location: attachment-storage.service.ts; s3.service.ts

Description: Auth-gated and size-capped (good), but MIME is client-declared; large default max increases abuse risk; no AV scanning observed.

Remediation: Constrain MIME on complete-upload; shorter presign TTL; per-user quotas; optional AV; keep buckets private.

Resolution: Deferred (P2). Uploads remain auth-gated and size-capped.

MEDIUM · F2 · No application security headers observed

Status: Partial

Location: Fastify app (@fastify/helmet)

Description: No CSP, HSTS, frame-ancestors, X-Content-Type-Options, or Referrer-Policy in app or sampled nginx configs.

Remediation: Add headers via @fastify/helmet; enable HSTS on HTTPS terminators.

Resolution: Helmet enabled (X-Content-Type-Options, frame options, Referrer-Policy; HSTS in production). CSP left disabled for static UI compatibility — tighten at CDN/nginx if needed.

MEDIUM · H1 · User content flows into AI prompts without hard isolation

Status: Open

Location: chat.service.ts; portal-ai prompts

Description: Custom/system prompt override and document HTML in review pipelines increase prompt-injection risk.

Remediation: Never let untrusted text set the system role; delimit untrusted input; treat model output as untrusted.

Resolution: Deferred (P2). Policy evaluation remains in place but is not a full isolation boundary.

MEDIUM · I2 · trustProxy: true without documented hop constraints

Status: Partial

Location: backend/src/server.ts

Description: Correct behind reverse proxy, but if Node is exposed directly, clients can spoof X-Forwarded-For.

Remediation: Expose Node only internally; configure trusted proxy count.

Resolution: Comment documents that Node must be internal-only behind the reverse proxy. Explicit hop-count config still recommended in deploy docs.

MEDIUM · J2 · Platform-wide backups contain full PII

Status: Partial

Location: admin-backup.service.ts

Description: JSON backups of all models are high-value data stores; combined with broad admin backup access, tenant isolation may fail.

Remediation: Encrypt backups at rest; super-admin only; tenant-scoped exports; retention limits.

Resolution: Access restricted to super-admin. Encryption at rest and retention policy still open (P3).

LOW · A8 · Weak password policy (length only)

Status: Partial

Location: backend/src/lib/password-policy.ts; auth.service.ts

Description: Minimum 8 characters; no complexity, breach-check, or lockout.

Remediation: Stronger policy; progressive lockout / CAPTCHA after failures.

Resolution: Minimum 10 characters with letter + number on register/reset/admin set-password. Progressive lockout/CAPTCHA still open.

Status: Fixed

Location: /api/health; /api/health/detail

Description: May return S3 enabled flag, bucket, endpoint, and error text.

Remediation: Public health returns { ok: true } only; detailed diagnostics behind admin auth.

Resolution: Public /api/health returns { ok: true }. Detail endpoint is super-admin only.

Status: Fixed

Location: attachment-storage.service.ts

Description: On MinIO failure, smaller files fell back into Mongo — increases DB exposure in dumps.

Remediation: Fail closed in production when S3 is required.

Resolution: Production no longer falls back to Mongo embedding when S3 is configured and upload fails.

Status: Fixed

Location: backend/src/server.ts

Description: Increased DoS memory pressure without rate limits.

Remediation: Lower global default; keep large bodies only on upload routes.

Resolution: Global bodyLimit is 2 MB. Document/dataset/portal import routes raise the limit locally (8–25 MB).

5. Remediation roadmap

- P0** DONE — Auth-protect/remove open /api/users*, dashboard, sessions, outputs, chat; require auth on AI routes; harden admin bootstrap; production CORS_ORIGIN
- P1** DONE — Super-admin backups; per-user chat isolation; rate limits; stop JWT in WS query; HTML/markdown sanitization (partial CSP remains)
- P2** OPEN — httpOnly cookie sessions + CSRF; token refresh/revocation; stronger upload MIME/AV checks; prompt-injection hard isolation; CSP at CDN
- P3** OPEN — Dependency scanning in CI; encrypt backups at rest; formal privacy review of historically exposed routes; progressive login lockout

6. Production deploy checklist

- Set AUTH_SECRET to a unique strong value (never the development default).
- Set CORS_ORIGIN to the production web origin allowlist.
- After first admin bootstrap: DEFAULT_ADMIN_ENABLED=false and rotate the bootstrap password.
- Expose the Node process only behind the reverse proxy (trustProxy assumes a trusted hop).
- If the API was public before remediation, review access logs for abuse of former open routes.
- Users must re-authenticate after deploy (24h JWT TTL + WebSocket auth change).

7. Methodology & limitations

Method: Static defensive code review of authentication, authorization, secrets handling, input/HTML rendering, uploads, CORS/headers, rate limiting, AI surfaces, and privacy-related APIs. Critical findings were spot-checked against source. The October remediation pass verified fixes in source (default-deny hook, removed /api/users*, bootstrap create-only, WS first-message auth, helmet/rate-limit, etc.).

Limitations: This is not a live penetration test, dynamic scanning campaign, or red-team engagement. No credentials were exercised against production, no exploits were developed, and secret values from .env were not read or included. Deployed runtime configuration may differ from source defaults.

8. Next steps

- Confirm production env: AUTH_SECRET, CORS_ORIGIN, DEFAULT_ADMIN_ENABLED=false after bootstrap.
- Add integration tests asserting 401/403 on unauthenticated calls to formerly open routes.
- Schedule P2 work: httpOnly sessions, refresh/revocation, upload MIME/AV, prompt isolation.
- Consider a professional third-party penetration test after P2, under a formal rules-of-engagement.
- Regenerate this PDF after further remediations: node docs/security/generate-security-audit-pdf.mjs